EAC1 — Jetpack Compose · Cicle de vida · ViewModel · NavHost

Kotlin + Compose · Android Studio · Emulador AVD

1 · ESTAT A COMPOSE

Gestión del estado reactivo en Compose y su persistencia ante recomposiciones y cambios de configuración.

```
var x = 0 // ✗ NO reactivo. Compose no ve los cambios de valor.
var x by remember { mutableStateOf(0) } // ✓ Sobrevive a recomposiciones. Muere al rotar pantalla.
var x by rememberSaveable { mutableStateOf(0) } // ✓✓ Sobrevive a rotaciones de pantalla y segundo
plano.
```

`remember { }` : Conserva el valor de la variable en memoria mientras el Composable siga en pantalla. Se destruye si la pantalla se recrea.

`rememberSaveable { }` : Guarda la variable en una estructura Bundle física. Sobrevive a rotaciones y recreación por falta de memoria.

`mutableStateOf(0)` : Envuelve el valor en un objeto de estado reactivo. Compose se suscribe a él y redibuja (recompone) la interfaz cuando cambia.

`by (delegación)` : Delega el acceso a la propiedad. Permite leer y escribir directamente en `x` sin tener que escribir `.value` en todo el código.

■ BUG: remember { mutableStateOf(0) } — las llaves { } del remember NO son opcionales.

2 · NAVEGACIÓ PER ESTAT (EAC1 EJ2 — SENSE NAVHOST)

Navegación manual condicional basada en el estado de una variable de progreso.

```
var pas by rememberSaveable { mutableStateOf(1) }
when (pas) {
    1 → PantallaNom(onNext = { pas = 2 })
    2 → PantallaQuantitat(onNext = { pas = 3 })
    3 → PantallaTipus(onNext = { pas = 4 })
    4 → PantallaResum(onReset = { pas = 1 })
}
```

`when (pas)` : Estructura de control que evalúa el estado del paso actual y renderiza dinámicamente un Composable u otro en función del valor.

`isNotBlank()` : Función estándar de Kotlin usada para validar si un campo de texto no está vacío o compuesto solo por espacios antes de avanzar de pantalla.

✓ Botó Següent desactivat: Button(enabled = text.isNotBlank())

3 · CICLE DE VIDA ACTIVITY — ORDRE DE CALLBACKS

Secuencias de ejecución y orden de los métodos del ciclo de vida de una Activity en Android.

```
Secuencias de callbacks:
Abrir app      : onCreate → onStart → onResume
Tancar app     : onPause → onStop → onDestroy
ROTAR PANTALLA : onPause → onStop → onDestroy → onCreate → onStart → onResume
Home (2n pla)  : onPause → onStop
Tornar de Home : onRestart → onStart → onResume
```

```
override fun onResume() {
    super.onResume() // ← SEMPRE cridar super.onX() primer!
    Log.d("MyTag", "onResume") // ← filtrar per tag al Logcat
}
```

`onCreate()` : Primer callback ejecutado al instanciar la Activity. Aquí se inicializan componentes y se define la UI inicial.

`onStart()` : Hace que la actividad sea visible en pantalla para el usuario.

`onResume()` : Sitúa la actividad en primer plano; es el momento en que se puede interactuar con ella.

`onPause()` : La actividad pierde el foco (ej. aparece un diálogo encima o se cambia a otra pantalla parcial).

`onStop()` : La actividad ya no es visible para el usuario (se ha cubierto por otra aplicación).

`onDestroy()` : Se invoca antes de destruir la actividad para liberar recursos pendientes.

`onRestart()` : Ejecutado tras pasar por onStop y antes de onStart cuando el usuario vuelve a la misma actividad.

`super.onX()` : Llama a la implementación base de Android para evitar que la aplicación falle al no completar la inicialización del sistema.

`Log.d(tag, msg)` : Registra mensajes de depuración en la consola Logcat de Android Studio de manera eficiente.

4 · VIEWMODEL

Gestión y almacenamiento seguro del estado de la UI persistiendo a cambios de configuración.

```
class PizzaViewModel : ViewModel() {
    var nom by mutableStateOf("") // reactivo
    var quantitat by mutableStateOf(1)
    var tipus by mutableStateOf("")
}

// A l'Activity o Composable:
val vm: PizzaViewModel = viewModel() // NO: PizzaViewModel()
```

`ViewModel()` : Clase base de Android que contiene y gestiona el estado de manera desacoplada del ciclo de vida de las Activities. Sobrevive a rotaciones.

`viewModel()` : Función de extensión que recupera una instancia de ViewModel ya existente o crea una nueva vinculada al ciclo de vida del propietario (Activity). Evita instanciar a mano con constructores vacíos directos.

■ BUG TEVES: CounterViewModel() directo — usa siempre la función delegada viewModel().

✓ El ViewModel vive ligado al ciclo de la Activity, no al Composable individual.

5 · NAVHOST + NAVCONTROLLER (EAC1 EJ3)

Navegación declarativa oficial de Jetpack Compose mediante rutas y control de pila.

```
val nav = rememberNavController()
NavHost(nav, startDestination = "inici") {
    composable("inici") { PantallaInici(nav, vm) }
    composable("comanda") { PantallaComanda(nav, vm) }
    composable("resum") { PantallaResum(nav, vm) }
    // Ruta amb argument:
    composable("detall/{cat}") { back →
        val cat = back.arguments?.getString("cat")
        PantallaDetall(cat, nav)
    }
}
// Navegar:
nav.navigate("comanda")
nav.navigate("detall/${categoria}") // interpolat, NO literal detall/{categoria}
nav.popBackStack() // tornar enrere
```

`rememberNavController()` : Crea y recuerda el gestor central de navegación que mantiene la pila de pantallas (Backstack).

`NavHost( ... )` : Contenedor que asocia un NavController con un grafo de navegación de pantallas.

`composable("route") { }` : Declara una pantalla en el grafo asignándole una clave/ruta en formato de texto.

`back.arguments?.getString(key)` : Obtiene los parámetros inyectados en la URL de navegación de la pantalla origen.

`nav.navigate("route")` : Empuja una nueva pantalla sobre la pila de navegación actual.

`nav.popBackStack()` : Destruye la pantalla superior de la pila para volver a la pantalla anterior.

■ BUG TEVES: navigate("detall/{42}") — usa interpolación $categoria, no literales con llaves { }.

6 · SCAFFOLD + TOPAPPBAR

Estructuras visuales estándar de diseño Material Design en Jetpack Compose.

```
Scaffold(
    topBar = {
        TopAppBar(
            title = { Text(titolActual) },
            navigationIcon = {
                IconButton(onClick = { nav.popBackStack() }) {
                    Icon(Icons.Default.ArrowBack, null)
                }
            }
        )
    }
) { padding →
    NavHost(modifier = Modifier.padding(padding), ... )
}
```

`Scaffold( ... )` : Composable contenedor que reserva zonas fijas (slots) para la barra superior, barra inferior, botones flotantes y contenido.

`TopAppBar( ... )` : Componente de barra de herramientas superior para títulos y menús.

`IconButton( ... ) { Icon( ... ) }` : Envuelve un icono vectorial en un botón táctil con área de click accesible.

`padding (PaddingValues)` : Margen interno que calcula automáticamente el Scaffold para que el contenido de la pantalla no quede solapado por las barras de herramientas. Es obligatorio aplicarlo en los modificadores hijos.

EAC2 — Retrofit (Chuck Norris API) + Room (Favorits)

Kotlin + Compose · Coroutines · KSP · Flow

1 · RETROFIT — DATA CLASS + INTERFÍCIE

Definición de modelos de datos e interfaces para la comunicación y consumo de APIs REST.

```
// Data class (mapeja el JSON de l'API):
data class Joke(
    val id: String,
    val value: String,
    val categories: List<String>
)

// Interfície de l'API:
interface ChuckNorrisApi {
    @GET("jokes/random")
    suspend fun getRandomJoke(): Joke

    @GET("jokes/categories")
    suspend fun getCategories(): List<String>

    @GET("jokes/random")
    suspend fun getJokeByCategory(
        @Query("category") cat: String
    ): Joke
}
```

`data class` : Tipo de clase Kotlin optimizada para almacenar datos. Genera automáticamente toString(), equals() y hashCode().

`@GET("endpoint")` : Anotación de Retrofit que define que el método realiza una petición HTTP GET al endpoint indicado.

`suspend fun` : Define funciones suspendibles ejecutadas de forma asíncrona dentro del marco de corrutinas de Kotlin sin bloquear el hilo de renderizado de la UI.

`@Query("name")` : Inyecta parámetros de consulta (Query parameters) en la ruta final de la URL (ej: `jokes/random?category=cat`).

2 · INSTÀNCIA RETROFIT (SINGLETON)

Definición de la instancia única del cliente HTTP Retrofit mediante patrón Singleton.

```
object RetrofitInstance {
    val api: ChuckNorrisApi by lazy {
        Retrofit.Builder()
            .baseUrl("https://api.chucknorris.io/")
            .addConverterFactory(GsonConverterFactory.create())
            .build()
            .create(ChuckNorrisApi::class.java)
    }
}
```

`object` : Palabra reservada de Kotlin que declara un Singleton seguro para hilos de manera automática.

`by lazy { }` : Retrasa la construcción del cliente Retrofit hasta la primera vez que se intente consumir la variable `api`, ahorrando recursos de carga inicial.

`.baseUrl( ... )` : Define la raíz URL de red donde se enviarán las peticiones del servicio de red.

`.addConverterFactory(GsonConverterFactory.create())` : Configura GSON como el deserializador que convertirá las cadenas de texto JSON en objetos `Joke` de Kotlin automáticamente.

`.create(Interface.class)` : Crea la implementación del código interno para las rutas descritas en la interfaz.

✓ AndroidManifest.xml requiere permiso de Internet: <uses-permission android:name="android.permission.INTERNET"/>

3 · RETROFIT AL VIEWMODEL AMB COROUTINES

Lanzamiento asíncrono y control seguro de estados de carga y error en llamadas a APIs.

```
class JokesViewModel : ViewModel() {
    var joke by mutableStateOf<Joke?>(null)
    var isLoading by mutableStateOf(false)
    var error by mutableStateOf<String?>(null)

    fun loadRandom() {
        viewModelScope.launch {
            isLoading = true
            error = null
            try {
                joke = RetrofitInstance.api.getRandomJoke()
            } catch (e: Exception) {
                error = e.message
            } finally {
                isLoading = false
            }
        }
    }
}
// A la UI: LaunchedEffect(Unit) { vm.loadRandom() }
```

`viewModelScope.launch { }` : Lanza una corrutina en el hilo principal acoplada al ciclo de vida del ViewModel. Cancela automáticamente las operaciones de red si el ViewModel se destruye.

`try/catch/finally` : Permite capturar errores de conexión (servidor caído, falta de wifi) y actualizar el estado de error de forma segura antes de apagar el indicador de carga.

`LaunchedEffect(Unit) { }` : Efecto de Compose que ejecuta la función asíncrona una única vez al pintarse el componente por primera vez.

4 · ROOM — ENTITY + DAO + DATABASE

Definición del mapeo relacional de base de datos local SQLite mediante anotaciones de Room.

```
@Entity(tableName = "favorite_jokes")
data class FavoriteJoke(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val apiId: String,
    val text: String,
    val category: String?
)

@Dao
interface FavoriteJokeDao {
    @Insert(onConflict = OnConflictStrategy.IGNORE)
    suspend fun insert(joke: FavoriteJoke)

    @Query("SELECT * FROM favorite_jokes")
    fun getAll(): Flow<List<FavoriteJoke>> // Flow = reactiu!

    @Delete
    suspend fun delete(joke: FavoriteJoke)
}
```

`@Entity(tableName = " ... ")` : Define que la clase Kotlin se convertirá en una tabla SQLite local dentro del almacenamiento.

`@PrimaryKey(autoGenerate = true)` : Indica el campo clave identificador primario incremental de la tabla.

`@Dao` : Indica una interfaz Data Access Object que expone las operaciones básicas de acceso a datos compiladas por Room.

`@Insert(onConflict = ... )` : Inserta datos. La estrategia `IGNORE` evita colisiones duplicadas si la clave primaria ya está registrada.

`@Query("SELECT * FROM ... ")` : Ejecuta sentencias SQL en crudo.

`Flow<List<T>>` : Tipo de flujo asíncrono y reactivo de Kotlin. Emite automáticamente la lista actualizada de elementos cuando ocurre cualquier cambio en las tablas de la BD.

`@Delete` : Borra el registro coincidente de la base de datos de manera atómica.

✓ Flow<List<T>> provoca que la UI se actualice sola de forma automática al ocurrir cambios.

5 · DATABASE SINGLETON

Configuración e instanciación segura de la base de datos local SQLite mediante Room.

```
@Database(entities = [FavoriteJoke::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun favoriteJokeDao(): FavoriteJokeDao

    companion object {
        @Volatile private var INST: AppDatabase? = null

        fun getInstance(ctx: Context) =
            INST ?: synchronized(this) {
                Room.databaseBuilder(ctx.applicationContext,
                    AppDatabase::class.java, "jokes_db")
                    .build().also { INST = it }
            }
    }
}
```

`@Database( ... )` : Define la base de datos principal declarando las tablas que la forman y su versión.

`@Volatile` : Garantiza que cualquier cambio en la variable `INST` sea visible instantáneamente en todos los hilos del procesador de forma segura.

`synchronized(this)` : Garantiza que solo un hilo a la vez pueda entrar al bloque de código de creación, previniendo duplicidades e inconsistencias en la base de datos local.

`Room.databaseBuilder( ... )` : Constructor que asocia el contexto físico, la clase base y el nombre de archivo ".db" para inicializar la base de datos SQLite en disco.

6 · ROOM AL VIEWMODEL + COLLECTASSTATE()

Suscripción reactiva a flujos de bases de datos desde el Composable UI.

```
// En el ViewModel:
val favoritesFlow = db.favoriteJokeDao().getAll()

// En el Composable:
val list by vm.favoritesFlow.collectAsState(initial = emptyList())
```

`collectAsState(initial)` : Transforma el flujo reactivo `Flow` en un objeto de estado observable por Compose. Cada nueva modificación en la base de datos actualiza el valor de `list` y recompone la pantalla.

7 · VERSIONS COMPATIBLES KSP (CRÍTIC)

Reglas críticas de compatibilidad de dependencias de compilación.

```
// libs.versions.toml:
kotlin = "2.1.0"
ksp = "2.1.0-1.0.29"

// gradle.properties:
android.disallowKotlinSourceSets=false
```

`ksp / kotlin compatibility` : KSP es el motor que procesa las anotaciones de Room en tiempo de compilación. Su versión debe coincidir exactamente con la versión de Kotlin utilizada en el proyecto para evitar errores fatales de compilación.

■ BUG TEVES: KSP/Kotlin incompatibles — Kotlin 2.1.0 requiere KSP versión exacta 2.1.0-1.0.29.

EAC3 — libGDX · Joc Submarinista · Kotlin

Game · Screen · Actor · TiledMap · Camera · Animation · Preferences

1 · ARQUITECTURA — JERARQUIA DE CLASSES

Estructura general de clases del framework de videojuegos libGDX y su flujo de datos.

```
SpaceRace : Game          // punt entrada → setScreen()
SplashScreen : Screen     // carrega assets
GameScreen : Screen       // render() + GameState
ScrollHandler : Group     // Background[] + Asteroid[]
Scrollable : Actor        // base: posicio + velocitat
Spacecraft/Diver : Actor // el jugador
AssetManager (object)     // textures, sons, fonts
Settings (object)         // constants GAME_WIDTH, etc.
```

`Game` : Clase principal que actúa como punto de arranque del ciclo de vida y controla la pila de pantallas mediante el cambio de estados de pantallas activas.

`Screen` : Interfaz que representa una escena única (menús principales, pantalla de carga de texturas, niveles de juego).

`Actor` : Componente básico 2D que vive dentro de una escena. Mantiene posición, tamaño, lógica de dibujado y escala propia.

`Group` : Subclase de Actor que puede contener múltiples actores hijos dentro de él para facilitar movimientos en grupo o capas.

`AssetManager` : Cargador centralizado que almacena y gestiona en memoria texturas y sonidos de forma eficiente evitando fugas de memoria GPU.

`setScreen(screen)` : Método heredado de `Game` que destruye la pantalla activa y carga el ciclo de vida de la nueva pantalla.

2 · GAMESCREEN — RENDER() + GAMESTATE

Bucle principal del motor de juego y control de flujo de estados de juego.

```
enum class GameState { READY, RUNNING, GAMEOVER }
var state = GameState.READY

override fun render(delta: Float) {
    stage.draw() // dibuixa tots els Actors
    when (state) {
        GameState.READY → updateReady()
        GameState.RUNNING → updateRunning(delta)
        GameState.GAMEOVER → updateGameOver(delta)
    }
}

fun updateRunning(delta: Float) {
    stage.act(delta) // actualitza tots els Actors
    survivalTime += delta // comptador temps
    oxygen -= DRAIN * delta // pèrdua progressiva
    if (oxygen ≤ 0f) state = GameState.GAMEOVER
}
```

`render(delta)` : Método del bucle de juego ejecutado automáticamente por el motor a unos 60 FPS. Se encarga de pintar la pantalla en cada fotograma.

`stage.draw()` : Ordena a todos los actores asociados al escenario que se dibujen a sí mismos.

`stage.act(delta)` : Dispara la actualización de la lógica de los actores (físicas, movimiento, inteligencia artificial).

`delta (Float)` : Tiempo transcurrido (en fracciones de segundo) desde el último frame. Vital para independizar la velocidad de la física del rendimiento de la máquina.

3 · ORTHOGRAPHICCAMERA + TILEDMAP

Gestión del viewport 2D mediante cámaras ortográficas y renderizado de mapas Tiled.

```
val cam = OrthographicCamera()
cam.setToOrtho(false, VPORT_W, VPORT_H)
val map = TmxMapLoader().load("maps/map.tmx")
val renderer = OrthogonalTiledMapRenderer(map, 1f)

fun updateCamera() {
    cam.position.x = diver.x + diver.width / 2
    cam.position.y = diver.y + diver.height / 2
    cam.position.x = cam.position.x.coerceIn(
        VPORT_W/2f, MAP_W - VPORT_W/2f
    )
    cam.position.y = cam.position.y.coerceIn(
        VPORT_H/2f, MAP_H - VPORT_H/2f
    )
    cam.update() // OBLIGATORI — recalcula matriu
}
// Al dibuixar:
renderer.setView(cam)
renderer.render()
```

`OrthographicCamera()` : Cámara 2D sin distorsión de perspectiva, ideal para videojuegos bidimensionales.

`setToOrtho(yDown, width, height)` : Configura el puerto de visión (viewport) y define la orientación del eje Y (si va hacia arriba o hacia abajo).

`TmxMapLoader().load( ... )` : Carga el archivo del mapa `.tmx` compilado externamente en Tiled.

`OrthogonalTiledMapRenderer( ... )` : Renderizador especializado en dibujar mapas en cuadrícula ortogonal.

`coerceIn(min, max)` : Restringe un valor flotante a un rango delimitado. Evita que la cámara se desplace fuera de los límites del mapa.

`cam.update()` : Recalcula la matriz de proyección de la cámara. Debe ejecutarse tras cualquier cambio en la posición de la cámara.

`renderer.setView(cam)` : Vincula la matriz de la cámara actual al renderizador de mapas para dibujar solo el área visible.

`renderer.render()` : Dibuja las capas de la cuadrícula de mosaicos del mapa cargado.

■ BUG: sin cam.update() el renderizado no reacciona al movimiento del personaje (se congela la pantalla).

4 · MOVIMENT FLOTACIÓ + COL·LISIONS ENTORN

Físicas de gravedad, impulsos y cálculo matemático de colisiones rectangulares.

```
var velY = 0f
val GRAVITY = -120f // força cap avall
val THRUST = 200f // quan jugador prem

override fun act(delta: Float) {
    super.act(delta)
    velY += GRAVITY * delta // gravetat sempre
    if (isBoosting) velY = THRUST
    y += velY * delta // mou personatge
    y = y.coerceIn(0f, MAP_H - height) // límits
}

// Col·lisió rectangles:
val diverR = Rectangle(x, y, width, height)
if (Intersector.overlaps(diverR, obstacleR)) {
    // col·lisió detectada
}
```

`super.act(delta)` : Ejecuta las acciones por defecto del Actor (necesario para que las animaciones internas sigan reproduciéndose).

`Rectangle(x, y, w, h)` : Crea una caja de colisión lógica paralela a los ejes para simular el espacio ocupado por un objeto.

`Intersector.overlaps(r1, r2)` : Comprueba eficientemente si dos rectángulos virtuales intersectan en coordenadas XY.

✓ Regla física obligatoria: y += vel * delta. Sin multiplicar por delta la velocidad del juego dependerá de los FPS del hardware.

5 · ANIMATIONS — ANIMATION<TEXTUREREGION>

Partición de hojas de spritesheets y renderizado dinámico de animaciones.

```
val sheet = Texture("diver.png")
val reg = TextureRegion.split(sheet, 32, 32)
val idle = Animation<TextureRegion>(
    0.15f,
    reg[0][0], reg[0][1], reg[0][2],
    playMode = Animation.PlayMode.LOOP
)
var stateTime = 0f

// Al draw():
stateTime += Gdx.graphics.deltaTime
val frame = when {
    velY > 20f → animUp.getKeyFrame(stateTime, true)
    velY < -20f → animDown.getKeyFrame(stateTime, true)
    else → animIdle.getKeyFrame(stateTime, true)
}
batch.draw(frame, x, y, width, height)
```

`Texture("path")` : Carga una imagen rasterizada desde la carpeta de assets directamente en la memoria gráfica VRAM.

`TextureRegion.split(sheet, w, h)` : Divide la textura de la hoja en una matriz bidimensional de regiones de fotogramas del tamaño indicado (ej: 32×32 píxeles).

`Animation(duration, frames ... )` : Gestor de animación que organiza la secuencia y duración de cada fotograma individual.

`stateTime += deltaTime` : Acumulador de tiempo de juego transcurrido utilizado para determinar qué fotograma debe mostrarse.

`getKeyFrame(time, loop)` : Obtiene la región correspondiente al momento actual de la reproducción de la animación.

`batch.draw(frame, ... )` : Dibuja el fotograma exacto en pantalla aplicando posiciones de escala y renderizado.

6 · MINES + EXPLOSIÓ + PREFERENCES

Gestión y control de persistencia de configuraciones locales.

```
// Guardar records:
val prefs = Gdx.app.getPreferences("MyPrefs")
prefs.putInteger("highScore", score)
prefs.flush() // ← guarda físicament al disc
```

`prefs.flush()` : Vuelca de manera segura e inmediata los datos cacheados en las preferencias locales hacia el almacenamiento físico en disco. Sin esto, los cambios se pierden al apagar la aplicación.

■ BUG TEVES: Daño de minas continuo en cada frame — el flag `mine.exploded = true` evita que dañe más de una vez.

■ BUG TEVES: Si olvidas prefs.flush(), las marcas de récord de puntuación se pierden tras cerrar la app.